# NATIONAL INSTITUTE OF TECHNOLOGY WARANGAL

Department of Mathematics

**Computer Programming and Data Structures with Python**
**Group Presentation Problems**

MA26003 / MA26005 Academic Year 2026–27

---

### Group 1: (Roll no: 1,2,3,4)

An inversion in a list is a pair of indices $i < j$ for which $a_i > a_j$.

(a) Implement merge sort using recursion. Show the successive splitting and merging of the example list.
(b) Write a direct method to count inversions using two loops. Then modify merge sort to count inversions during the merging process.
(c) Compare the running times of the two counting methods and explain how inversions are counted across the two halves.
(d) Find the number of inversions in a list of $n$ distinct integers when the elements are arranged in increasing order and in decreasing order.

*Example.* For `[2, 4, 1, 3, 5]`, the sorted list is `[1, 2, 3, 4, 5]` and the inversion count is 3.

### Group 2: (Roll no: 5,6,7,8

A sorted list of distinct integers has been rotated at an unknown position. For example, `[15, 18, 2, 3, 6, 12]`.

(a) Write a function that searches for a given element. Return its index, or $-1$ if absent.
(b) Trace the search by recording the low, middle and high indices after each step.
(c) Explain why one of the two halves must be sorted at every step.
(d) Write a Python function to find the largest and smallest elements in a given list without using the built-in `max()` and `min()` functions.

*Example.* For `[15, 18, 2, 3, 6, 12]`, searching for `3` returns index `3`; searching for `7` returns `-1`.

### Group 3: (Roll no: 9,10,11,12)

For each element of a list, find the first element to its right that is strictly greater. Write $-1$ if no such element exists.

(a) Solve the problem by checking the elements to the right of each position.
(b) Solve it again using a stack represented by a Python list.
(c) Trace the stack operations for the example and compare the two methods in running time.
(d) Implement the basic stack operations `push`, `pop`, and `peek` using a Python list. Explain how an empty stack is handled.

*Example.* Input `[4, 5, 2, 10, 8]` gives `[5, 10, 10, -1, -1]`.

### Group 4: (Roll no: 13,14,15,16)

People numbered $1, 2, \ldots, n$ stand in a circle. Starting from person 1, every $k$th remaining person is removed. Counting then resumes with the next person.

(a) Simulate the process with a Python list. Print the elimination order and the survivor.
(b) Derive or explain a recurrence for the survivor's position, and implement it recursively or iteratively.
(c) Compare the two methods and test what happens when $k = 1$ or $n = 1$.
(d) Explain how the modulo operator is used to move circularly through a list.

*Example.* For $n = 7$ and $k = 3$, the removal order is $3, 6, 2, 7, 5, 1$; person 4 survives.

**Group 5: (Roll no: 17,18,19,20)**

A substring consists of consecutive characters of a string. Find a longest substring in which no character occurs more than once.

(a) Write a straightforward solution that considers substrings and checks for repetitions.
(b) Develop a more efficient method using two indices and a dictionary storing the last position of each character.
(c) Show how the indices change for the example. Compare the running times, and test an empty string and a string of identical characters.
(d) Write a Python function to determine whether all characters in a given string are distinct. Use a dictionary or a set.

*Example.* For `abcaefbb`, one longest substring is `bcaef`, of length 5.

**Group 6: (Roll no: 21,22,23,24)**

An interval $[a, b]$ contains all values between $a$ and $b$, including the endpoints. Given a list of intervals, combine every pair of intervals that overlap. Intervals sharing an endpoint are also considered overlapping.

(a) Write a Python function that returns the merged intervals in increasing order of starting point.
(b) Explain why sorting by starting point makes it possible to process the intervals in a single subsequent scan.
(c) Trace your method, state its running time, and test intervals that are nested or share endpoints.
(d) Write a Python function to determine whether two given intervals overlap. Include the case where the intervals share an endpoint.

*Example.* `[[1,3],[2,6],[8,10],[9,12]]` becomes `[[1,6],[8,12]]`.

**Group 7: (Roll no: 25,26,27,28)**

Represent a matrix by a nested Python list; do not use NumPy for the main algorithm.

(a) Write a function that rotates an $m \times n$ matrix $90^\circ$ clockwise, producing an $n \times m$ matrix.
(b) For a square matrix, perform the same rotation in place, without creating a second matrix of the same size.
(c) Explain how an entry's row and column indices change, and compare the extra space required by the two methods.

*Example.* $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \longrightarrow \begin{bmatrix} 7 & 4 & 1 \\ 8 & 5 & 2 \\ 9 & 6 & 3 \end{bmatrix}$.

**Group 8: (Roll no: 29,30)**

Given distinct positive integers and a target $S$, find all subsets whose elements add up to $S$. Each element may be used at most once.

(a) Write a recursive program that includes or excludes each element and prints every valid subset exactly once.
(b) Explain how a subset is formed by including or excluding elements from a list. Illustrate using the given example.
(c) Modify the program to count the number of subsets whose sum equals the target $S$. Test the program with a case where no solution exists.

*Example.* For `[3, 4, 5, 6]` and $S = 9$, the solutions are `[3, 6]` and `[4, 5]`.